Dublin Bikes — Project Report

UCD School of Computer Science — Software Engineering Project

Kaiwen Yao • Tzuyu Chang • Ziling Huang April 2026

Authors and Contributions — each author contributed an agreed equal share of **33.33%**.

Author	%	Type of contribution
Kaiwen Yao	33.33	**Code** (`scraper`: standalone data collection service, `main_scraper.py` periodic loop with retry logic, `fetch_stations.py` JCDecaux API ingest, `fetch_weather.py` weather ingest, SQLAlchemy ORM models, Docker containerisation, Jenkins CI/CD pipeline for image build and EC2 deployment; `flask-app`: application factory pattern, environment-based config with strict validation, Flask-Migrate for schema management, JWT-based authentication API with registration/login/logout/access–refresh token rotation and email verification flow, Pydantic request/response contracts, Station API (list all stations, 24 h availability history, latest snapshot, bike-availability prediction endpoint), AI Chat API integrating Alibaba Cloud Qwen LLM via LangChain with dual response modes—standard JSON and real-time SSE streaming—and persistent session management secured by JWT middleware; `machine_learning`: full ML pipeline (EDA, feature engineering, correlation analysis), trained and compared Linear Regression, Decision Tree, Random Forest, and Gradient Boosting models, selected Random Forest as best performer, exported model and feature artefacts, published to Hugging Face (`ucdse/bike_availability_model`) with Jenkins pipeline pulling model at build time; Testing: built pytest infrastructure with SQLite in-memory isolation and shared fixtures, wrote unit and integration tests for user auth, station API, prediction service, AI chat, contracts, schemas, and utilities); **Report** (Machine Learning Model, Process); **Management** (CI/CD, Docker, Scrum Master Sprint 3).
Tzuyu Chang	33.33	**Code** (`react-app`: bootstrapped React + TypeScript + Vite project, configured routing with `react-router-dom`, Tailwind CSS, ESLint, and TypeScript config; `Layout.tsx` top-level layout wrapper, `Header.tsx` site-wide navigation bar, `Footer.tsx` site-wide footer; `Home.tsx` landing page with project introduction and quick navigation; Maps page integrating Google Maps JavaScript API for interactive bike-station map with real-time station markers, availability badges, and station detail panel showing current and historical availability; Authentication pages—`Register.tsx` registration form with client-side validation, `Login.tsx` login form with JWT token storage, `VerifyEmail.tsx` prompt page after registration, `Activate.tsx` email token activation handler; Profile page displaying user info from `/api/users/me`, supporting profile update and logout; `flask-app`: designed and implemented `GET /api/stations/status` real-time station status endpoint using an efficient SQLAlchemy subquery with `func.max()` grouped by station number to retrieve only the most recent record per station, avoiding full-table scans); **Report** (Overview, Requirements, Testing); **Management** (User Research—designed and conducted user persona interviews, authored full interview script covering commuting habits, pain points, and feature expectations, synthesised findings into user personas; Scrum Master Sprint 1 & 4).

Author	%	Type of contribution
Ziling Huang	33.33	**Code** (UI/UX Design: high-fidelity Figma mockups for Home, Maps, Chat, Profile, and authentication flows, defined visual language—colour palette, typography, component hierarchy, responsive layout grid—and produced annotated wireframes as blueprint for frontend implementation; `flask-app`: provisioned and configured Amazon RDS MySQL instance including security group rules, subnet configuration, connection parameter tuning, and environment-variable credential management, validated schema compatibility between Flask-Migrate migrations and RDS; Weather API—`GET /api/weather` fetching hourly and daily forecast from OpenWeatherMap One Call API, Pydantic `WeatherQueryDTO`/`WeatherDataVO` for strict request validation and response serialisation, error handling for invalid coordinates and API failures; Journey/Route Planning API—`POST /api/journey/plan` accepting text addresses or coordinates, Google Maps Geocoding integration, server-side optimal route calculation (nearest available source station → nearest free-dock destination station), `api_retry` decorator for resilient Google Maps API calls; `react-app`: reusable Weather Component consuming `/api/weather` displaying current conditions, hourly and daily forecast across Maps and Home pages; Chat Page with full chat UI, message history, typing indicators, SSE streaming display, and session persistence across page reloads connected to `/api/chat` and `/api/chat/stream`; Journey Planning UI within Maps page with journey search form, rendering recommended pick-up and drop-off stations on the map, and displaying estimated route duration; Testing: wrote unit and integration tests for weather and journey APIs with mocked Google Maps API); **Report** (Architecture and Design); **Management** (AWS RDS, UI/UX Design, Scrum Master Sprint 2).

Screen Recording (6–8 min max). URL: *TODO — paste final video URL.* 3–5 min: feature walkthrough with voice, browser URL bar clearly showing the **EC2 address**. ∼3 min: each member presents **1 min** on their contributions (Kaiwen → Tzuyu → Ziling).

GitHub Project.
Docs/report: `https://github.com/ucdse/docs` · Backend: `https://github.com/ucdse/flask-app` · Frontend: `https://github.com/ucdse/react-app` · Scraper: `https://github.com/ucdse/scraper`

Product & Sprint Backlogs.
Product backlog: *TODO — URL.* Sprint 1: *TODO.* Sprint 2: *TODO.* Sprint 3: *TODO.* Sprint 4: *TODO.*

Generative AI Chats (one per student, in the `docs` repo.)
KaiwenYao_Generative_AI_chats.md
TzuyuChang_Generative_AI_chats.md
ZilingHuang_Generative_AI_chats.md